Østfold University College

A Systematic Literature Review of DevOps in quantum software engineering

Systematic Literature Review

Kadir Amini, Berat Murseli

Avdeling for Informasjonsteknologi
Høgskolen i Østfold
Halden
May 9, 2026

A Systematic Literature Review of DevOps in quantum software engineering

Systematic Literature Review

Kadir Amini, Berat Murseli

Avdeling for Informasjonsteknologi
Høgskolen i Østfold
Halden
May 9, 2026

Abstract

Acknowledgments

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Quantum computing has developed rapidly in recent years, with cloud-accessible quantum processors now available from IBM, Google, Amazon, and Microsoft. However, the software engineering side of the field is still relatively immature. Much of the existing research focuses on algorithms, hardware, and performance, while less attention has been given to development workflows, testing pipelines, deployment practices, and automation in quantum software projects.

DevOps has quickly become a dominant part in software engineers ability to deliver software quickly and reliably. continuous integration (CI), automated testing, containerized deployment, and more are standard practices for software development. If quantum software is ever to go beyond science research and become deployable technology, the ability to deliver, maintain, and develop it will require DevOps.

quantum computers are different from classical computers at a fundamental level. Quantum computer are probabilistic due to relying on quantum mechanics to represent the binary values. Quantum computers are sensitive to noise, qubit calibration, decoherence, gate errors, SDK verion, and more. All this results in the same program potentially behaving differently on when and where it is executed.

1.1 Goal

A key objective of this SLR is to identify research gaps in the field of quantum computing, more specifically within DevOps and CI/CD. By identifying gaps in the existing research, this paper aims to provide direction for future studies and contribute to helping quantum computing researchers move closer to practical solutions in this field.

A second objective is to provide a theoretical framework for how DevOps and CI/CD practices can be integrated into the field of quantum computing. This aims to provide an understanding of how development, testing, deployment, and continuous integration processes can be adapted to make use of the strengths of quantum software development. By doing so, this paper contributes ideas on how quantum computing can reduce the gap between concept and reality.

1.2 Background

The purpose of this project is to gain a better understanding of how quantum computing works. With the advanced technology available today, we wanted to explore how modern

computing technology is built, what principles lie behind the development of quantum computing, and how this emerging field may influence future technological development.

Understanding the concept of quantum computing is a large field, and it would take too much time to cover everything. Therefore, we chose to focus more on the field of DevOps and CI/CD, especially understanding the fundamentals behind the integration of quantum computing. We wanted to explore how existing applications are adapting to and using this technology today, and whether practical use cases already exist.

Spørsmål til lører: trenger vi info om quantom software i bunn?

1.3 Research Questions

This review is guided by the following research questions:

- **RQ1:** How does quantum computing DevOps compare to standard DevOps?
- **RQ2:** What challenges exist in implementing continuous integration and continuous delivery for quantum software?
- **RQ3:** What are the main barriers preventing the adoption of standard DevOps practices in quantum software engineering?

These questions allow the review to move beyond a simple mapping of papers. They make it possible to compare quantum software workflows with classical DevOps, identify technical and organizational challenges, and explain why adoption is currently limited.

Chapter 2

Related Work

Before initiating the SLR we find it greatly important to explore the landscape of the current state of research within quantum computing. The landscape of quantum software engineering is rapidly advancing as such reviewing related literature will help give insight into the field. The main goal was to find if there have been done any SLR with a similar scope before. The secondary goal is to find useful context for this study.

2.1 Quantum Software Engineering

Quantum software engineering (QSE) has become a sub discipline of software engineering, with the goal of adapting engineering practices into quantum computers. Several foundational works from around 2020 set out principles for the discipline. The Talavera Manifesto, produced at the first QANSWER workshop, lists principles and calls for action including dedicated methodologies, reuse, debugging support, and lifecycle models for quantum software [1]. QANSWER was a international workshop on Quantum Software Engineering and Programming, held in Talavera de la Reina, Spain. The workshop gathered researchers and industry practitioners to discuss how existing software engineering processes, methods, and practices could be applied or adapted to the development of quantum software, and where entirely new ones might be needed. QANSWER is an early case of the QSE field organizing itself to work on shared principles.

Since the talavera manifesto more papers have been published. A paper from 2024 [2] provides a survey of QSE and the quantum software development lifecycle, mapping existing SDKs, programming languages, and proposed lifecycle models. Serrano, Pérez-Castillo, and Piattini [3] edited a Springer volume that brings together formal methods, modeling languages, and reengineering approaches for quantum software. Across these works it is shown that quantum programs differ from classical because their behavior is probabilistic and sensitive. Hardware noise, calibration, decoherence, gate fidelity, and SDK version all can affect the result. Established software engineering practices cannot simply be transferred because of this. They must instead be adapted to a setting where the same program may produce different outputs depending on when and where it is executed.

2.2 DevOps Classical Software Engineering

In 2007 Patrick Debois noticed that development and operations teams were not working well together. The gaps and conflicts that arose were unsettling to Debois. That unsettling

feeling was the spark which would later motivate the development of DevOps [4]. DevOps is about improving the workflow of the development and operations in a project. This is done by following certain practices in terms of planing, coding, testing, and deployment. Finally Continuous integration and continuous delivery (CI/CD) make up the backbone of DevOps [5].

In 2017, Shahin et al. conducted an SLR where 69 studies were reviewed. The SLR classified approaches, tools, challenges, and reported benefits related to continuous integration, continuous delivery, and continuous deployment [6].

Soares et al. later conducted an SLR focusing on the effects of continuous integration on software development. This was done by taking 101 empirical studies and showing that CI is associated with shorter feedback cycles, improved build health, and faster defect detection. They also cover challenges connected to CI like, long build times, unstable builds, and flaky tests [7].

There have also been studies that explore more broadly on DevOps as a software engineering practice. Faustino et al. conducted an SLR on the benefits of DevOps, where he found multiple advantages. collaboration increased, faster delivery, increased automation, and better software quality [8]. Azad and Hyrynsalmi later studied success factors for DevOps adoption. They highlight that organizational culture, collaboration, automation, management support, and suitable tooling as the core factors for DevOps implementation [9].

From all this it can be understood that DevOps is built with the goal of making software delivery automated, repeatable, and reliable. This is done by adding build processes, automated tests, and deployment pipelines that are designed for fast and reproducible feedback. Translating this into QSE presents a challenge due to the fact that quantum programs are probabilistic and sensitive.

2.3 Existing similar SLRs

There have been studies on QSE however. to our knowledge, there have been no published SLR that directly focuses on DevOps. We have been able to find papers that are closely related.

AlSanad and Akbar have a SLR which is similar to ours as they focus on DevOps enablers for QSE [10]. This Paper directly connects DevOps to QSE and discuss DevOps adoption. Unfortunately this is a preprint which has not been peer reviewed. As a result we mention it in the related work, but we can not use it for our SLR as it has not been peer reviewed.

Since there has not been a peer reviewed paper which has covered this topic, we believe this paper is crucial for advancing QSE. As has been covered further previously, DevOps is an integral part of software engineering and has many advantages. If quantum computers are to be used in any practical applications having DevOps will be crucial.

Chapter 3

Methodology

Systematic literature reviews are useful in software engineering for identifying, evaluating and synthesizing existing research on a topic. The purpose of this study is to investigate DevOps in Quantum software engineering. An SLR will give a structured way to search for relevant studies and select the right studies to finally assess and synthesize.

This SLR follows the guidelines proposed by Kitchenham [11], which are widely used for SLRs in software engineering. The Guidelines split up the process into the main phases. planning the review, conducting the review, and reporting the results. Following these phases will give structure to the review and help with repeatability and less bias.

3.1 Review Design

This project follows a systematic literature review methodology consisting of planning to get better control of the structure, then conducting and reporting phases to get an overview of the existing research. This follows Kitchenham's [11] guidelines, which emphasize a transparent, systematic, and structured review process to reduce bias.

The planning phase established the research questions, the search string, the inclusion and exclusion criteria, the quality-assessment instrument, and the data-extraction template. The conducting phase executed searches on the six databases as chosen, applied the criteria in two screening stages for the first initial search, and ran one round of backward snowballing on the most-cited studies. Then, three stages were carried out, adding quality assessment and scoring, and then extracting data synthesis.

3.2 PICOC framing

PICOC stands for Population, Intervention, Comparison, Outcome, and Context. These elements provide structure and help define what the review is about, what type of information we are trying to gather, and what we want to compare based on the research topic.

In our review, we chose to use these elements to help justify the research focus and identify the most important information based on the available papers. This helps improve the quality of the review and ensures that the selected studies are relevant for future work.

This is our PICOC table:

Element	Definition for this review
Population	Quantum software projects, quantum programs, developers, and teams using quantum SDKs or quantum development workflows.
Intervention	DevOps practices, CI/CD, workflow automation, testing pipelines, version control, reproducibility, and related engineering methods.
Comparison	Standard DevOps or classical software engineering practices when a comparison is possible.
Outcome	Productivity, reliability, maintainability, automation level, correctness, reproducibility, and identified barriers.
Context	Quantum software engineering in academia or industry, including simulator-based and hardware-based settings.

Table 3.1: PICOC table

3.3 Search strategy

To cover the topics of quantum software and quantum computing, we selected several academic databases for our systematic literature review. These databases were chosen because they provide access to high-quality, peer-reviewed research relevant to our study.

- **ACM**
- **IEEE Xplore**
- **Science Direct**
- **Springer Link**
- **Wiley**
- **Taylor And Francis**

We started with Google Scholar and tested different search strings. Several of the first search strings returned a very large number of papers because they were too broad. Therefore, we made the search string more specific and tested different versions until we found a more complete search string that returned around 4,000 papers. This gave us a better basis for finding relevant and high-quality papers. After that, we used the chosen search string to search in the different databases we had selected.

Our selected search string is: **(“quantum software” OR “quantum program” OR “quantum programming” OR “quantum computing”) AND (“DevOps” OR “CI/CD”)**

3.4 Inclusion and exclusion criteria

As part of our review method, we needed to define criteria for what should be included and excluded in our SLR. Since our language knowledge is mainly based on English, we chose

to include only papers written in English and exclude papers written in other languages. This makes it possible for us to read the papers and understand the information.

We also chose to include only peer-reviewed papers, as this helps ensure a higher level of quality. Papers that did not meet the required quality level were excluded.

There is a large amount of research on quantum computing, and some papers focus mainly on quantum physics. These papers were excluded because our review focuses specifically on quantum software development and software engineering.

We wanted to focus on newer research in the field, so we excluded papers published before 2020. This helps ensure that the review includes more recent developments and innovations.

To make our research more relevant to the search field, we also excluded papers that did not focus on topics such as DevOps, CI/CD, workflow automation, and testing pipelines. This helped us narrow the review and identify papers that were more closely related to our research topic.

Inclusion criteria	**Exclusion criteria**
Written in English	Non-English publications
Peer-reviewed articles or clearly relevant scholarly papers	Non-peer-reviewed material with no academic value
Focuses on quantum software development or quantum software engineering	Pure quantum physics or hardware papers with no software relevance
Published between 2020 and 2026	Published before 2020 unless needed for background only
Discusses DevOps, CI/CD, workflow automation, testing pipelines, or closely related practices in a quantum context	No meaningful connection to DevOps or software-engineering practice in a quantum context

Table 3.2: Inclusion and Exclusion Criteria Table

3.5 Study selection process

For the study selection process, we started with title and abstract screening to remove irrelevant studies. This means that we went through all the papers we found and reviewed their titles and abstracts to remove studies that were not related to our topic. After the first stage, we conducted full-text screening using inclusion and exclusion criteria to make the selection more relevant. Then, we carried out a quality assessment to ensure that the chosen papers were of high quality and suitable for our research. In the final stage, we performed data extraction and synthesis as the end result. This process follows the guidelines by [11]

3.6 Quality assessment

Each research paper that passed full-text screening was then quality assessed with a given score based on the quality assessment questions, which were partly implemented from Kitchenham's QA checklist [11]. As this study is a qualitative paper, we also adopted some of the QA questions from the qualitative study checklist.

QA #	Question	Scoring
QA1	Are the research aims clearly stated and relevant to quantum software engineering, DevOps, CI/CD, testing, or automation?	Yes (1) / Partly (0.5) / No (0)
QA2	How credible is the paper?	Yes / Partly / No
QA3	How well is has diversity off perspective and context been explored in the paper?	Yes / Partly / No
QA4	Does the study include validation, prototype, experiment, simulation, CI/CD pipeline	Yes / Partly / No
QA5	Does the study discuss limitations, challenges, or barriers to adoption?	Yes / Partly / No

Table 3.3: Quality assessment questions and scoring

The maximum score for each study was 5. Each quality assessment question was scored as Yes = 1, Partly = 0.5, and No = 0. The scores were used to give an overall quality of the selected papers and to support the final selection of studies. Studies with lower scores were considered less reliable and were not prioritized in the synthesis.

3.7 Data extraction plan

The purpose of the data extraction plan is to ensure that the same information is collected from each paper and that the extraction process is consistent for all selected studies. This helps ensure that relevant information is extracted while reducing the risk of missing important details.

In the extraction fields, we included author, year, venue, paper type, and database source. These fields help ensure that the extracted information is relevant to the research questions of the review. They also make it easier to describe trends among the papers, such as publication year, which was important for assessing the relevance of each paper.

The second field was included to ensure that the extracted information was related to the main topic of the research. This included papers that discuss workflow, testing, automation, deployment, version control, and CI/CD. This field was highly important because it helped determine whether each paper was relevant to the core focus of the review.

The contribution type was used to classify whether the paper presented a tool, framework, method, benchmark, or review. This helped us understand the type of contribution each paper made and compare the papers more systematically.

To further assess the quality of the papers, the evidence type was also extracted. This showed how the claims in each paper were supported. The evidence types included experiments, case studies, benchmarks, and surveys. This field was useful for identifying whether the paper was practical, empirical, or conceptual.

The main findings were key result related to research questions 1, 2, and 3. This ensured that the selected papers were aligned with the scope of the review and made it easier to identify similarities and differences between the papers.

We also included challenges and barriers, such as technical, organizational, tooling, reproducibility, and skill-related barriers. In addition, a quality score was added based on

the quality checklist. This helped strengthen the assessment of each paper and made the review more reliable.

Overall, the data extraction plan provided a clear and structured way to collect and organize information from the selected papers. It helped ensure that the extracted data was strongly related to the research questions and supported a systematic analysis of the literature.

Field	**Purpose**
Paper metadata	Authors, year, venue, paper type, and database source.
Quantum software focus	Whether the paper discusses workflow, testing, automation, deployment, version control, or CI/CD.
Contribution type	Tool, framework, method, benchmark, review.
Evidence type	Experiment, case study, benchmark, survey, position paper.
Main findings	Key results related to RQ1, RQ2, or RQ3.
Barriers/challenges	Technical, organizational, tooling, reproducibility, hardware-access, or skills-related barriers.
Quality score	Assessment result from the quality checklist.

Table 3.4: Data extraction table

3.8 Data synthesis plan

After the data was extracted, as described above, we analyzed the data to answer the research questions. Our SLR is qualitative and focuses on DevOps and CI/CD in quantum software engineering. This means that we will group and compare the findings rather than analyze them statistically.

After the data extraction, the information will first be organized according to the three research questions. Findings about the comparison between quantum DevOps and standard DevOps will be placed under RQ1. Findings about CI/CD, automation, testing, deployment, and reproducibility will be placed under RQ2. Findings about barriers such as hardware access, lack of tools, organizational challenges, lack of skills, and technical limitations will be placed under RQ3.

The next step will be to identify common themes across the selected papers. For example, if several papers discuss problems with quantum hardware access, this will be treated as a common theme. The same will be done for topics such as testing, reproducibility, simulator differences, and unstable results. This makes it easier to identify patterns and understand which challenges are most common in the literature.

The synthesis will also compare the type of contribution each paper makes. Some papers may present tools or frameworks, while others may present methods, benchmarks, case studies, or reviews. This helps show whether the field is mainly practical, theoretical, or still in an early stage.

The evidence type and quality score of each paper will also be considered. Papers with stronger evidence, such as experiments, benchmarks, prototypes, or case studies, will be given more weight than papers with weaker evidence. The quality score will help support the final conclusions and make the review more reliable.

Finally, the results will be presented in the results chapter and organized according to the research questions. Where useful, tables may be used to summarize and compare the findings. Overall, this synthesis plan provides a clear way to move from findings in individual papers to broader conclusions about DevOps and CI/CD in quantum software engineering.

Chapter 4

Results

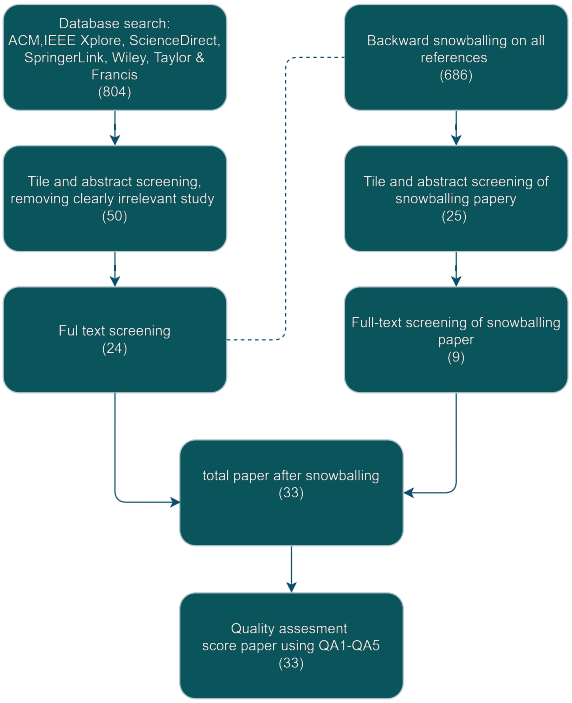

Figure 4.1: Methodology Illustration

4.1 Overview of Included Studies

As a result of screening, inclusion and exclusion, and quality assessment, we ended up with a total of 33 papers in the final synthesis. These papers were published between 2020 and 2026. From Table 4.1 it is evident that there has been an increase in papers interested in DevOps within the QSE field The final result for the 33 studies was 17 journal articles, 15 conference papers, and one book section. This result indicates that the main methods for this topic are journal articles and conference papers, with journal articles having two more papers than conference papers.

Table 4.2 shows that the included studies had varying levels of quality. The results show that the scores ranged from 1.5 to 5, with an average score of 3.82. A total of 27

Publication year	Number of studies
2020	3
2021	2
2022	2
2023	4
2024	12
2025	6
2026	4
Total	**33**

Table 4.1: Distribution of included studies by publication year

studies scored 3 or higher, of which 18 scored 4 or higher. This indicates that there is strong evidence based on the criteria set for the quality assessment, such as clear information about quantum software engineering, highly credible papers, and good discussion of challenges and limitations.

Quality score category	Number of studies
4.5–5.0	16
4.0–4.4	2
3.0–3.9	9
Below 3.0	6
Total	**33**

Table 4.2: Quality score distribution of included studies

As shown in Table 4.3, out of the 33 papers, there are 25 studies that cover all three research questions. This means that they discuss differences between quantum and classical DevOps, CI/CD or automation challenges, and barriers to adoption. This also indicates that there is a clear connection between these topics. Many of the papers discuss DevOps practices with varying degrees of quality related to the research questions.

Research question coverage	Number of studies
RQ1, RQ2, and RQ3	25
RQ1 and RQ3	3
RQ2 and RQ3	2
RQ3 only	2
RQ1 and RQ2	1
Total	**33**

Table 4.3: Coverage of research questions in the included studies

Chapter 5

Discussion

5.1 Summary

Chapter 6

Conclusion

Bibliography

[1] M. Piattini, G. Peterssen, R. Pérez-Castillo, J. L. Hevia, M. A. Serrano, G. Hernández, I. García Rodríguez de Guzmán, C. A. Paradela, M. Polo, E. Murina, L. Jiménez, J. C. Marqueño, R. Gallego, J. Tura, F. Phillipson, J. M. Murillo, A. Niño, and M. Rodríguez, "The Talavera manifesto for quantum software engineering and programming," in *Proceedings of the 1st International Workshop on the QuANtum SoftWare Engineering & pRogramming (QANSWER 2020)*, vol. 2561 of *CEUR Workshop Proceedings*, (Talavera de la Reina, Spain), pp. 1–5, 2020.

[2] K. Dwivedi, M. Haghparast, and T. Mikkonen, "Quantum software engineering and quantum software development lifecycle: a survey," *Cluster Computing*, 2024.

[3] M. A. Serrano, R. Pérez-Castillo, and M. Piattini, eds., *Quantum Software Engineering*. Berlin: Springer, 2022.

[4] A. Iheanacho, "A brief history of devops and its impact on software development," Feb. 2023.

[5] C. Ebert, G. Gallardo, J. Hernantes, and N. Serrano, "Devops," *IEEE Software*, vol. 33, no. 3, pp. 94–100, 2016.

[6] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[7] E. Soares, G. Sizílio, J. Santos, D. A. da Costa, and U. Kulesza, "The effects of continuous integration on software development: A systematic literature review," *Empirical Software Engineering*, vol. 27, no. 3, p. 78, 2022.

[8] J. Faustino, D. Adriano, R. Amaro, R. Pereira, and M. M. da Silva, "Devops benefits: A systematic literature review," *Software: Practice and Experience*, vol. 52, no. 9, pp. 1905–1926, 2022.

[9] N. Azad and S. Hyrynsalmi, "Devops critical success factors — a systematic literature review," *Information and Software Technology*, vol. 157, p. 107150, 2023.

[10] A. AlSanad and M. Akbar, "Optimizing devops enablers for quantum software development." Preprint, Research Square, Nov. 2023. Version 1, posted 14 November 2023. Not peer reviewed.

[11] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Technical Report EBSE-2007-01, EBSE, 2007.